

DSL DE TESTES UNITÁRIOS PARA LINGUAGEM FUNCIONAL 3 (LF3)

Equipe: Ericlecio Thiago e Anderson M. Marinho

O PROBLEMA

1. Comportamento Fail-Fast (o interpretador morre no primeiro erro).
2. Impossibilidade de testar "Caminhos Errado" (erros esperados).

```
let
  var sacar = fn saldo valor . if saldo > valor then saldo - valor else saldo - "erro"
in
  print( sacar (100, 20) == 80 ) ,

  sacar(10, 50) , 2. Tentativa de testar Caminho Triste (Fail-Fast)

  print( sacar(200, 50) == 150 ) - 3. Esse teste final NUNCA vai ser executado
```

A SOLUÇÃO

- DSL Nativa na AST inspirada no mercado (Jest, HUnit).
- Padrão BDD (Behavior-Driven Development).
- Execução Isolada (Non-blocking).

GRAMÁTICA (BNF)

- ExpDescribe ::= "describe" "(" Expressao "," ListExp ")"
- ExpTest ::= "test" "(" Expressao "," Expressao ")"
- ExpExpect ::= "expect" "(" Expressao "," Expressao ")"
- ExpToThrow ::= "toThrow" "(" Expressao ")"

ENGENHARIA DA AST (CLASSES JAVA)

- Funcional3.jj: Tokens mapeados sem recursividade infinita.
- ExpDescribe, ExpTest, ExpExpect: Nós da AST (o ExpTest é o escudo).
- ExpToThrow: Nó de semântica invertida.
- TesteFalhouException: Exceção customizada.
- GerenciadorDeTestes: O Singleton (placar e JSON).

EXEMPLO PRÁTICO (BDD)

```
describe("Operacao de Saque no Caixa Eletronico",
  test("Dado saldo positivo, Quando sacar, Entao deve liberar",
    expect(sacar(100, 20), 80)
  ),
  test("Dado saldo insuficiente, Entao deve gerar erro",
    toThrow(sacar(10, 50))
  )
)
```

RESULTADOS

EXEMPLO DE RELATÓRIO DE TESTES:

Operacao de Saque
Total Executados: 2
Passaram: 2
Falharam: 0

PRÓXIMOS PASSOS

- ☐ **Relatório consolidado:** confirmar se describe(...) sempre imprime um resumo e se inclui detalhes de falhas quando existirem (ExpDescribe + GerenciadorDeTestes).
- ☐ **expect(a, b):** checar se o equals() dos Valor* realmente representa igualdade “da linguagem” (ex.: inteiros/booleanos/strings/listas) e se a mensagem de falha está ok.
- ☐ **toThrow(expr):** confirmar se “exceção nativa esperada” é realmente a definição desejada (hoje ele considera sucesso ao capturar qualquer Exception, exceto TesteFalhouException).
- ☐ **Tipos:** garantir que describe retorna STRING e test/expect/toThrow retornam BOOLEANO (já codificado assim).

PRÓXIMOS PASSOS

- ☐ **Exemplos reais:** revisar Funcional3/input e quaisquer arquivos de teste existentes para ver se batem com a gramática e com o comportamento esperado.
- ☐ **Cobertura de cenários:** pelo menos 1 exemplo de sucesso (expect) e 1 de erro (toThrow) exercitando erro de tipagem/escopo, como descrito no README.
- ☐ **Identificação do teste:** garantir que ExpTest sempre usa a descrição correta ao registrar falhas/sucessos.
- ☐ **Erros “nativos não tratados”:** revisar o texto gerado para ficar consistente e útil.

PRÓXIMOS PASSOS



Tokens e precedência: garantir que describe/test/expect/toThrow não conflitam com outras expressões e que são parseadas onde se espera (Funcional3.jj adiciona as regras em PExpUnaria()).



Forma aceita: checar se describe("...", test(...), test(...)) aceita lista com vírgulas e sem trailing comma (parece que não aceita vírgula final, o que pode afetar exemplos).



reset() no describe: hoje ExpDescribe sempre reseta antes de rodar; avaliar se isso impede múltiplos describe no mesmo programa (se isso for desejado, ok; se quiser agregação global, seria ajuste).



**OBRIGADO PELA
ATENÇÃO!**